SOFTWARE ENGINEERING

Submission 3

Operator Fleet & Incident

Smart E-Mobility Hub / VNU-HCM Urban Area

Mai Hoàng Phương Đông — 2452265

Individual design contribution and shared architecture integration

8 November 2026

Includes deployment and implementation views, class design, 22 method descriptions and 14 bonus acceptance test specifications.

Design submission for student and team review. The report does not claim a running integrated backend or executed backend tests.

Contents

Submission 3 - Fleet & Incident 3

Deployment view 5

Development / Implementation view 6

Fleet & Incident class diagram 7

Fleet & Incident - Class and Method Design 8

Fleet & Incident - Test Cases (Bonus) 18

Submission 3 - Fleet & Incident

Author: Mai Hoang Phuong Dong - 2452265. Module owner: member 4.

This is the individual design contribution for Operator Fleet & Incident, including its assigned telemetry scope and the member-4 integration contribution to the team's architectural views. It builds on the submitted Submission 2 sections 1.4, 2.4, 3.4 and 4.4, plus the external telemetry flows in sections 2.7 and 3.7.

Deliverable index

Deliverable	Editable source / output
Individual report	Submission3_Fleet_Incident_2452265.pdf
Deployment view contribution	PlantUML, PNG, SVG
Team Implementation view consolidation	PlantUML, PNG, SVG
Individual class diagram	PlantUML, PNG, SVG
Complete method catalogue	22 operation descriptions and data contracts
Bonus test cases	14 acceptance test specifications
Shared design agreement	Architecture baseline

Scope and architectural contribution

Deployment: Student and Operator browsers communicate with a single application/API process. A mock telemetry producer sends timestamped vehicle, parking and charging-point observations. FleetService, IncidentService, TelemetryAdapter and an internal health scheduler run inside the same process as the other five domain modules. JSON fixtures initialize authoritative in-memory state. The deployment diagram shows host/runtime/artifact placement; in-process dependencies are described by the Implementation view. The simulator may share the application laptop during the demonstration.

Implementation: presentation invokes authorized API handlers, which route to the six owning modules. Fleet aggregates Hub/resource data, coordinates transfers via Rental and Parking contracts, and handles point incidents via Charging. TelemetryAdapter validates intake then delegates reconciliation to each resource owner. Repository/Unit of Work contracts publish one consistent state and result record per command/event. What-if consumes a copied versioned snapshot; its recommendations do not modify live state.

The views are a proposed group design contribution, ready for review by the other owners. They are not evidence of a running integrated backend. Existing Submission 2 screens remain browser mockups; the current src and tests scaffolds are not expanded by this design task.

Individual class design

Five owned classes cover the module: FleetService, IncidentService, TelemetryAdapter, RedistributionOrder and IncidentTicket. IncidentService is the one added service relative to the original four-class assignment; it keeps repair/restoration separate from transfer coordination. The architecture baseline records that ownership change.

The diagram shows attributes, 22 typed operations, entity associations and multiplicities. External classes MobilityHub, SharedVehicle and ChargingPoint retain their original owners. ParkingSession remains accessed through the Parking contract described in the method catalogue. IncidentTicket targets exactly one vehicle or charging point. An order keeps historical assigned IDs and a subset of confirmed arrivals; a vehicle may not have incompatible concurrent active

commitments.

All class-diagram operations have exactly one method description. Data types, states, atomicity, request/event deduplication and error rules are defined alongside the catalogue. Method names refine the Submission 2 controller messages into application services without changing the core business rules.

Critical behavior preserved from Submissions 1 and 2

• Dashboard queries are read-only and expose missing/stale data.

• Creating an assignment reserves resources but does not change physical location or occupancy.

• Only confirmed new arrivals change vehicle location and Hub counts; partial arrival and safe retries are supported.

• Faults restrict new allocations and flag affected incomplete transfers. Repair evidence and current owner checks are required before restoration.

• Telemetry cannot overwrite a valid booking/session, infer physical transfer completion or clear an unresolved incident from an idle report.

• All effects of a command/event, including its deduplication result, commit together within the single-process MVP.

• Background imbalance rules create alerts/recommendations, not live redistribution orders.

Review handoff and limitations

Member 1 reviews Hub counters and the shared Deployment view. Member 2 reviews vehicle commitments and restriction/restoration. Member 3 reviews capacity and parking observation reconciliation. Member 5 reviews charging-point/session evidence and recovery. Member 6 reviews snapshot isolation and UC-to-method traceability. Proposed strict event ordering and freshness boundaries must use the same conventions across owners.

The 14 bonus cases specify success, rejection, concurrency, retry, failure, incident recurrence and cross-module effects. They are design-level acceptance specifications; backend execution remains future implementation work. In-memory state and deduplication records last for one process lifetime; a restart reloads fixtures. Durable recovery would require a persistent implementation of the same transaction contract.

Sources and AI contribution

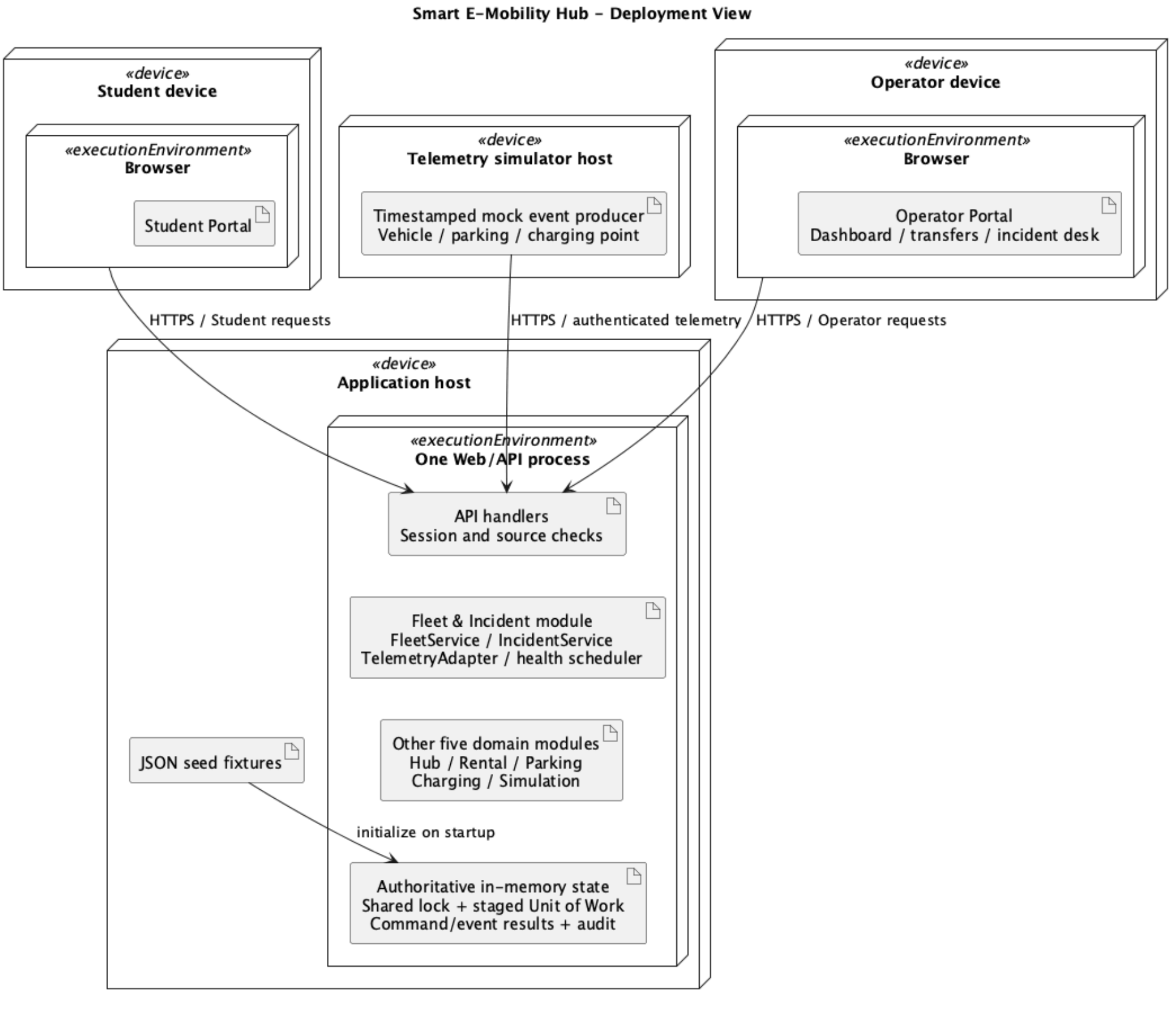
Sources: course brief, page 5 (Submission 3 deliverables); submitted Submission 2 report (Fleet and external telemetry sections); repository use-case catalogue, Operator Fleet & Incident scenarios, external_iot scenarios, non-interactive functional requirements, Submission 2 fleet sequence/state diagrams, and the Submission 3 task assignment and architecture baseline.

Generative AI disclosure: OpenAI Codex was used to assist with reviewing project materials, drafting and refining design documentation, and checking diagram rendering and method coverage. The student and team remain responsible for reviewing and understanding the submitted work.

Rebuild and verification

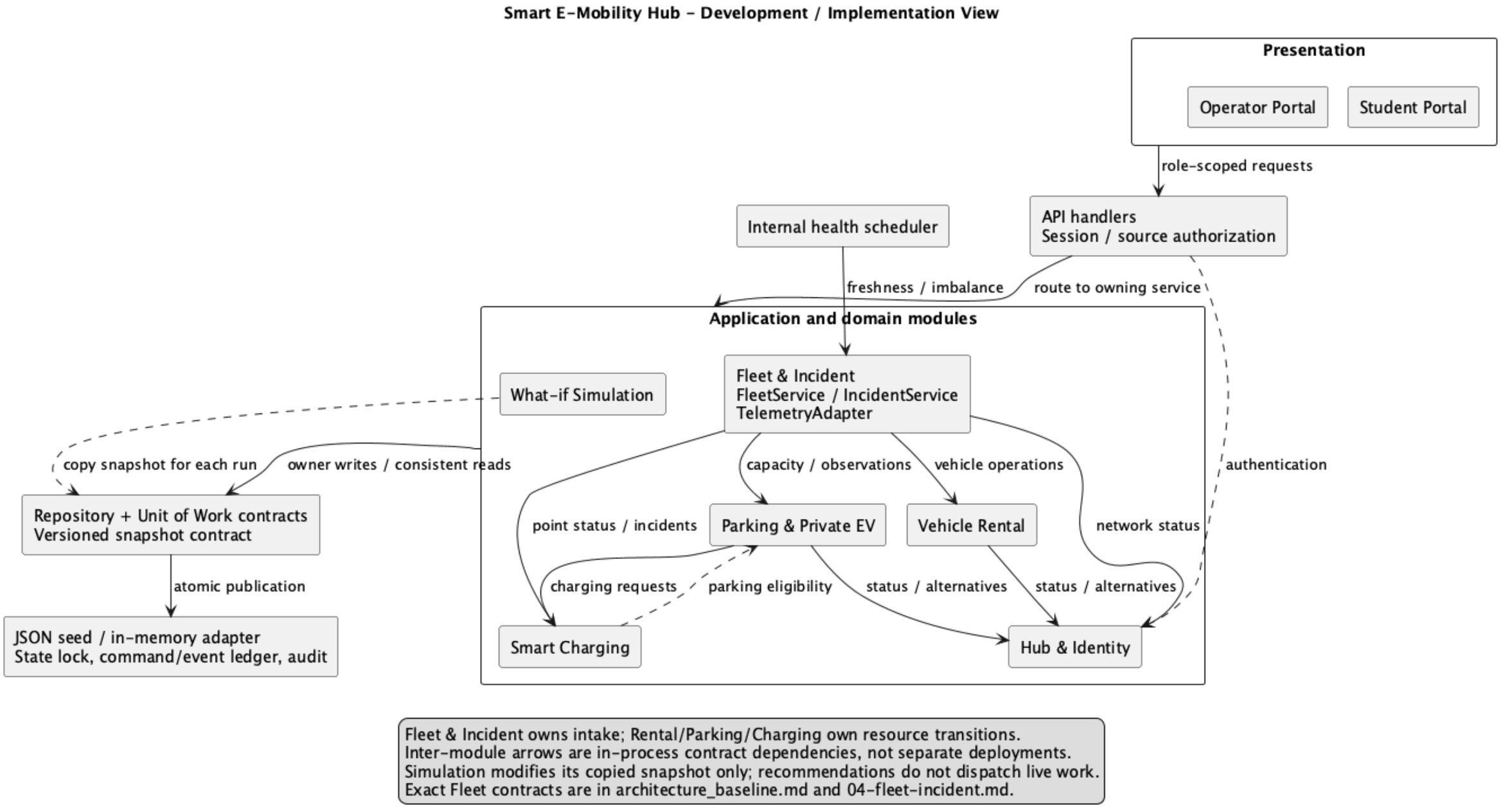
Render the three referenced .puml files with PlantUML using -tpng and -tsvg. Then run `python3 docs/submission-3/tools/build_fleet_report.py` from the repository root with reportlab, Pillow and pypdf installed. The builder checks exact one-to-one operation coverage before exporting the PDF to output/pdf. It uses Arial from the macOS system font directory, with a ReportLab font fallback on other systems. Diagram pages use A3 sheets to preserve label readability; text pages use A4.

Deployment view

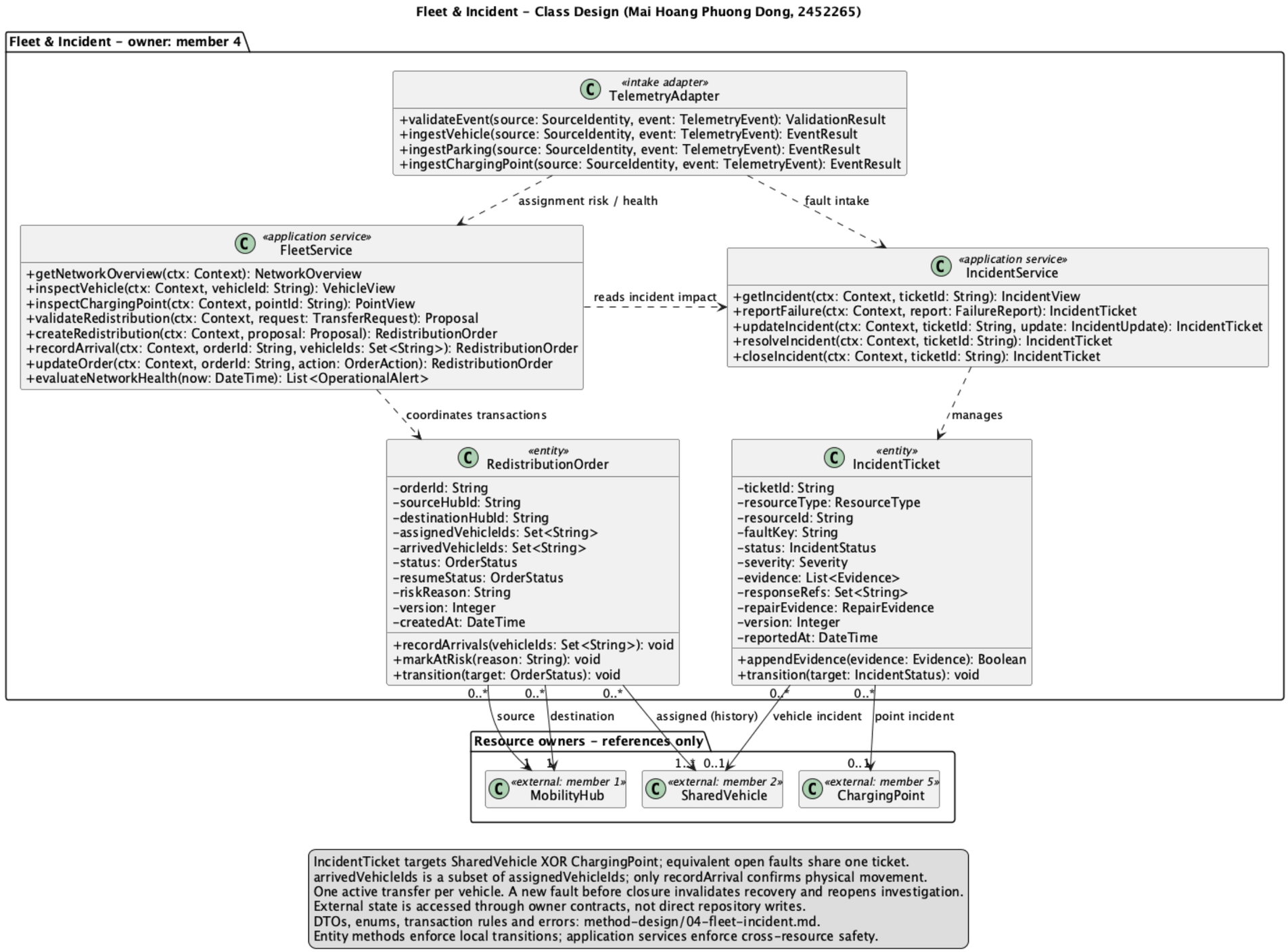


Artifacts inside the Web/API process use in-process calls and the same state lock.
Browser state holds UI selections; resource commitments are decided by the application.
No production database or physical IoT gateway is required for this MVP design.
State and deduplication results last for the process lifetime; restart reloads fixtures.
The simulator and application may share one laptop during demonstration.

Development / Implementation view



Fleet & Incident class diagram



IncidentTicket targets SharedVehicle XOR ChargingPoint; equivalent open faults share one ticket.
arrivedVehicleIds is a subset of assignedVehicleIds; only recordArrival confirms physical movement.
One active transfer per vehicle. A new fault before closure invalidates recovery and reopens investigation.
External state is accessed through owner contracts, not direct repository writes.
DTOs, enums, transaction rules and errors: method-design/04-fleet-incident.md.
Entity methods enforce local transitions; application services enforce cross-resource safety.

Fleet & Incident - Class and Method Design

Owner: Mai Hoang Phuong Dong - 2452265 (member 4).

Class diagram: 04-fleet-incident.puml.

Scope: UC-F01-F05, UC-I01-I03, UC-T01-T03 and UC-S03; background support for FR-NI-01/02/03/05/06/07/08. UC-I01 is abstract. IncidentService handles its vehicle and charging-point specializations using a typed resource reference. Rental, parking and charging policies remain owned by their respective modules.

1. Design decisions and class responsibilities

• FleetService coordinates read-only monitoring and redistribution transactions. It is also the internal entry point for periodic freshness and imbalance evaluation.

• IncidentService is an added member-4 service separating investigation, repair and restoration from fleet movement. This refines the initial FleetService proposal without adding a new team module.

• TelemetryAdapter authenticates event sources, validates/deduplicates observations and routes accepted updates through Rental, Parking or Charging contracts.

• RedistributionOrder owns assignment progress and risk state. IncidentTicket owns fault evidence, response references and its lifecycle. Neither entity directly changes another module's resource.

• API controllers from Submission 2 remain thin boundary handlers. FleetDashboardController and RedistributionController delegate to FleetService; IncidentController delegates to IncidentService; TelemetryController delegates to TelemetryAdapter. The former FeasibilityService responsibility is included in FleetService.validateRedistribution. This is a design refinement, with the same UC-S03 checks.

• The initial in-browser mockup is a UI prototype. This design places authoritative decisions in one application process as agreed in the architecture baseline.

2. Common contracts

Authorization, atomicity and retries

All public Context-based queries require a valid Operator session, except reportFailure, which additionally accepts a server-authenticated Student report for their own relevant rental or a trusted internal telemetry identity. Public callers cannot grant themselves an internal identity. evaluateNetworkHealth is an internal scheduled/domain-event operation. Entity methods are not public API endpoints.

Every mutation uses a requestId scoped to its authenticated principal and operation, plus a canonical payload hash. Same key and same payload returns the original committed result. Reusing a key with different content produces RequestConflict. Writes to existing orders/tickets require expectedVersion; check idempotency before checking that version so a committed retry still succeeds. Empty or malformed IDs, missing fields and invalid enums fail validation before mutation.

The MVP uses a single shared in-memory Unit of Work. A mutation acquires the application state lock, reads/revalidates owner state, stages all changes on a working copy and publishes the copy with its result record and audit entry only if every check succeeds. Exceptions discard the staged copy. Rental, Hub, Parking and Charging contracts participate in this same unit; no network transaction across independent services is assumed. A lost acknowledgement is resolved by retrying the original key. In-memory results are retained for the application lifetime; restart/durable recovery is outside this MVP and requires persistent storage if added later.

An assignment creates vehicle commitments and incoming capacity holds, not physical movement. Arrival confirmation changes confirmed Hub/location and occupancy exactly once. Telemetry stores observed location separately: it cannot confirm UC-F05 implicitly. Hub counters are projections of owner state and are refreshed in the same commit, rather than being independent competing sources of truth.

Types used by the class diagram

Type	Definition and required fields
Context	Server-verified principalId, role/source scope, requestId for writes, expectedVersion for existing entities; authorization derives from the authenticated session, not a browser role field.
TransferRequest	sourceHubId, destinationHubId, nonempty unique vehicleIds and intendedArrivalAt used for owner capacity/commitment checks. Source and destination differ.
Proposal	Original request, validationId, observedAt, resourceVersions, accepted/rejected vehicle IDs with reasons, feasible flag. Only an entirely feasible selected set is confirmable; a revised subset requires fresh validation.
OrderAction	DISPATCH, REVALIDATE or CANCEL. Cancellation releases only outstanding commitments; it never reverses confirmed arrivals.
NetworkOverview	Timestamped Hub/resource counts, open incident summaries, imbalance alerts and per-source freshness/unavailable flags.
VehicleView / PointView	Resource identity, latest condition and accepted observation time, confirmed location/Hub, operational restrictions and owner commitments; vehicle battery or point session/schedule details as appropriate.
IncidentView	Ticket, fault evidence, corrective action history, affected commitments and available permitted actions.
FailureReport	resourceType, resourceId, faultKey, severity and Evidence. A Student can report only a vehicle problem within their authorized rental context.
IncidentUpdate	action (ACCEPT, PLAN, START_REPAIR, PROGRESS, RECORD_RESPONSE), note, evidence/response reference, optional assignee, optional completed RepairEvidence. Required fields depend on action. It cannot directly resolve/close a ticket.
Evidence	evidenceId, source reference, observedAt and description/payload hash; an identical evidence ID/payload is appended once.
RepairEvidence	Recorded inspection/recovery result, actor, completion time, supporting reference and covered fault-evidence version. New fault evidence invalidates an earlier recovery assessment.
SourceIdentity	Authenticated providerId and permitted resource/reporting scope, established by the gateway.
TelemetryEvent	provider eventId, resource type/ID, Hub mapping, observedAt and typed payload. Vehicle: battery 0-100, observed location, condition. Parking: configured aggregate count or space status. Charging: point condition and session/vehicle references for session events.
ValidationResult	VALID, DUPLICATE or REJECTED, reason, canonical payload hash, observation scope and prior result when duplicated. Validation itself has no side effects.
EventResult	APPLIED, DUPLICATE, REJECTED or RECONCILIATION_REQUIRED; event reference, accepted time, affected references and reason. Retryable storage/conflict failures are errors rather than false APPLIED results.
OperationalAlert	Stable key, resource/order reference, rule, reason, createdAt, updatedAt and active/resolved state. Repeated checks update the same active alert.

Type	Definition and required fields
DateTime / collections	Compare timezone-aware instants; display GMT+7. Set values are unique; List preserves order. All returned mutable domain data is a copy/read model, not a live mutable store reference.

States and invariants

OrderStatus: Assigned, InTransit, PartiallyCompleted, AtRisk, Completed, Cancelled. IncidentStatus: Reported, Investigating, ActionAssigned, InRepair, Resolved, Closed. ResourceType: VEHICLE or CHARGING_POINT. Severity: LOW, MEDIUM or HIGH. These retain the Submission 2 names; the same incident guards are applied to charging-point incidents as a Submission 3 generalization.

• Order creation starts at Assigned. DISPATCH permits Assigned to InTransit. Newly verified arrivals calculate PartiallyCompleted or Completed from the arrived set. AtRisk preserves resumeStatus; successful revalidation restores Assigned, InTransit or PartiallyCompleted as appropriate. Cancellation is allowed before departure from Assigned, or from AtRisk after all unarrived vehicles have verified disposition and remaining holds can be released safely. Completed/Cancelled are terminal. A physically unlocated/in-transit vehicle must be reconciled before cancellation releases its commitment.

• Incidents progress Reported to Investigating to ActionAssigned to InRepair, with progress self-updates. Verified recovery permits ActionAssigned/InRepair to Resolved. Only Resolved can close. A new matching fault before closure reopens Resolved to Investigating and invalidates repair clearance; a fault after Closed creates a new ticket.

• Only IncidentService.resolveIncident can coordinate restoration. Repair evidence, current condition, all other unresolved faults and affected owner commitments are checked under the same state lock. Removing this incident's restriction is not equivalent to setting a reserved/rented/assigned vehicle Available. The owner derives its eligible state from all remaining commitments.

• Equivalent incident means the same resourceType + resourceId + faultKey. Distinct faults may coexist; resolving one must not remove another's restriction. A student report and telemetry evidence for the same fault are linked when their normalized faultKey matches.

• Staleness and imbalance thresholds are configuration, not UI constants. At now - observedAt >= staleInterval an observation is stale (boundary convention chosen for this design). Imbalance evaluation ignores unavailable/stale capacity for actionable recommendations and instead raises a data-quality alert.

3. FleetService methods

FleetService.getNetworkOverview(ctx: Context): NetworkOverview

Purpose: UC-F01: aggregate network status.

Input and output: Operator context; returns counts, alerts, incident summaries and source times.

Preconditions: Valid Operator access; upstream sources may be partly unavailable.

Postconditions / state change: Read-only consistent snapshot. Missing data is marked unavailable; old data is marked stale rather than replaced with zero.

Errors / exception result: Unauthorized: reject. Partial retrieval: return available fields with flags; total failure: DataUnavailable.

FleetService.inspectVehicle(ctx: Context, vehicleId: String): VehicleView

Purpose: UC-F02: inspect a vehicle before a decision.

Input and output: Context and vehicle ID; returns Rental state plus incident/freshness information.

Preconditions: Authorized Operator; known vehicle.

Postconditions / state change: No mutation; observed and confirmed location are distinguishable.

Errors / exception result: NotFound, Unauthorized; missing telemetry is explicitly unavailable.

FleetService.inspectChargingPoint(ctx: Context, pointId: String): PointView

Purpose: UC-F03: inspect a charging point.

Input and output: Context and point ID; returns Charging condition, current session and future commitments.

Preconditions: Authorized Operator; known point.

Postconditions / state change: Read-only; an idle observation never hides a future allocation or unresolved restriction.

Errors / exception result: NotFound, Unauthorized, DataUnavailable or partial status with flags.

FleetService.validateRedistribution(ctx: Context, request: TransferRequest): Proposal

Purpose: UC-F04 includes UC-S03 feasibility validation.

Input and output: Requested vehicles, two Hubs and arrival time; returns versioned proposal and reasons.

Preconditions: Operator; valid distinct Hubs and nonempty unique IDs.

Postconditions / state change: Read-only. Check source membership, safe condition, rental/reservation/assignment conflicts, source/destination operation and freshness, and destination capacity after existing occupancy and relevant commitments.

Errors / exception result: InvalidInput; ineligible IDs/insufficient capacity produce infeasible proposal with reasons. Unavailable or stale input prevents confirmation.

FleetService.createRedistribution(ctx: Context, proposal: Proposal): RedistributionOrder

Purpose: UC-F04: commit an approved proposal.

Input and output: Operator write context and feasible proposal; returns Assigned order.

Preconditions: Revalidate all selected vehicles, owner versions, Hub capacity and commitments under the lock.

Postconditions / state change: Atomically save order, vehicle commitments, destination holds, projections and idempotent result. Confirmed location and physical occupancy remain unchanged.

Errors / exception result: Conflict or stale proposal: reject and request revalidation. SaveFailure: publish nothing. Identical retry returns original order.

FleetService.recordArrival(ctx: Context, orderId: String, vehicleIds: Set<String>): RedistributionOrder

Purpose: UC-F05: confirm actual arrival, including a subset.

Input and output: Operator context/version and newly arrived vehicle IDs; returns updated order.

Preconditions: Incomplete non-AtRisk order; nonempty subset of assigned, not-yet-arrived vehicles; identity, physical arrival, restrictions and destination capacity verified with owners.

Postconditions / state change: In one commit update confirmed location, consume each incoming hold, release source occupancy, add destination occupancy, release transfer commitment, derive vehicle availability and call recordArrivals. No other vehicle moves.

Errors / exception result: Mismatch/duplicate ID, unsafe state, capacity conflict, VersionConflict: reject whole submitted subset. A retry of the same successful request returns original result. Any save failure rolls back all effects.

FleetService.updateOrder(ctx: Context, orderId: String, action: OrderAction): RedistributionOrder

Purpose: Record dispatch, recover an at-risk order or cancel remaining work.

Input and output: Context/version, ID and action; returns current order.

Preconditions: DISPATCH: Assigned. REVALIDATE: AtRisk and all outstanding resource/capacity constraints checked. CANCEL: allowed lifecycle state and verified disposition of every outstanding vehicle.

Postconditions / state change: Dispatch changes order only. Revalidation restores stored progress state. Cancellation releases only outstanding vehicle/capacity holds; arrived vehicles and historical occupancy remain unchanged.

Errors / exception result: InvalidTransition, unresolved conflict, missing disposition or VersionConflict: retain state and holds. SaveFailure rolls back.

FleetService.evaluateNetworkHealth(now: DateTime): List<OperationalAlert>

Purpose: FR-NI-03/06/07: scheduled/event-driven health checks.

Input and output: Trusted scheduler time; returns current changed/active alerts.

Preconditions: Internal invocation with configured thresholds and owner snapshots.

Postconditions / state change: Mark stale projections; upsert shortage/surplus/capacity/data-quality alerts. Flag incomplete orders whose outstanding vehicles now conflict, storing resumeStatus. Resolve an alert only when verified current data clears it. Do not create orders, restore AtRisk orders or move vehicles automatically.

Errors / exception result: Missing data produces a data-quality alert, not invented availability. Failed update is retried without duplicate alerts.

4. IncidentService methods

IncidentService.getIncident(ctx: Context, ticketId: String): IncidentView

Purpose: UC-I02/I03: review evidence and impact.

Input and output: Operator context and ticket ID; returns ticket and owner commitments.

Preconditions: Authorized Operator; known ticket.

Postconditions / state change: Read-only; viewing does not implicitly accept or resolve the incident.

Errors / exception result: Unauthorized, NotFound; unavailable owner state is flagged and restoration remains blocked.

IncidentService.reportFailure(ctx: Context, report: FailureReport): IncidentTicket

Purpose: Receive a valid report or a validated telemetry fault.

Input and output: Authorized source context and typed fault report; returns existing/new ticket.

Preconditions: Source may report this resource; evidence/resource mapping valid. Internal calls use the parent telemetry unit of work.

Postconditions / state change: Atomically create Reported ticket or append evidence once to equivalent non-Closed ticket, restrict allocation via owner, flag affected transfers AtRisk and upsert alerts. Recurrence after Resolved reopens Investigating and invalidates repair clearance; after Closed creates a new ticket. A point fault does not fabricate a stopped session.

Errors / exception result: InvalidInput, Unauthorized, RequestConflict; failure discards all incident, restriction and alert effects.

IncidentService.updateIncident(ctx: Context, ticketId: String, update: IncidentUpdate): IncidentTicket

Purpose: Record investigation, action plan, repair and affected-user response.

Input and output: Operator context/version and structured update; returns updated ticket.

Preconditions: ACCEPT from Reported; PLAN from Investigating; START_REPAIR from ActionAssigned; PROGRESS in InRepair; RECORD_RESPONSE in an unresolved ticket. Valid action-specific evidence/assignee/owner response reference.

Postconditions / state change: Transition through the permitted lifecycle, append audit/evidence and retain owner response references. Completed repair evidence is recorded without restoring eligibility. A note alone does not cancel a rental/reservation/session: the owning module must execute and confirm that action.

Errors / exception result: InvalidTransition, missing evidence, unverified owner response or VersionConflict: no mutation.

IncidentService.resolveIncident(ctx: Context, ticketId: String): IncidentTicket

Purpose: UC-I02/I03: verify recovery and coordinate restoration.

Input and output: Operator context/version and ID; returns Resolved ticket.

Preconditions: ActionAssigned/InRepair; complete repair/recovery evidence covers latest fault; live owner condition verifiable; affected commitments addressed and no conflicting fault remains.

Postconditions / state change: Atomically record resolution, remove only this ticket's restriction through Rental/Charging, recompute eligibility and alerts. Valid future booking/assignment stays committed; new capacity is exposed only when owner checks permit it.

Errors / exception result: RepairIncomplete, UnsafeResource, UnresolvedCommitment, DataUnavailable or VersionConflict: keep ticket unresolved and resource restricted. SaveFailure rolls back both sides.

IncidentService.closeIncident(ctx: Context, ticketId: String): IncidentTicket

Purpose: Close the resolved incident's administrative workflow.

Input and output: Operator context/version and ID; returns Closed ticket.

Preconditions: Resolved; no newer unreconciled fault evidence.

Postconditions / state change: Close ticket and preserve evidence/audit history; no additional allocation or occupancy effect.

Errors / exception result: InvalidTransition, newer fault or VersionConflict: reject. Identical retry returns original result.

5. TelemetryAdapter methods

TelemetryAdapter.validateEvent(source: SourceIdentity, event: TelemetryEvent): ValidationResult

Purpose: Shared source/payload/ordering validation for UC-T01-T03.

Input and output: Authenticated source and event; returns validation decision, reason and original result for a duplicate.

Preconditions: Gateway identity available; resource mappings, reporting mode and allowed clock skew configured.

Postconditions / state change: Read-only. Check scope, fields/ranges, resource mapping, known event key/hash and latest accepted observation. Exact duplicate returns prior result even after newer observations; reused event ID with different payload rejects. Non-duplicate older/equal timestamps and excessive future skew reject under this design's strict ordering rule.

Errors / exception result: Invalid source/scope: reject without protected details. Invalid/stale event: REJECTED. Missing state: retryable DataUnavailable; do not advance freshness.

TelemetryAdapter.ingestVehicle(source: SourceIdentity, event: TelemetryEvent): EventResult

Purpose: UC-T01 and FR-NI-01/02/07: apply vehicle observations.

Input and output: Vehicle event; returns event result with incident/order references.

Preconditions: validateEvent succeeds; recheck under transaction lock against latest Rental state and event ledger.

Postconditions / state change: Store observed location/battery/condition/freshness through Rental contract. Keep confirmed transfer location separate. Fault invokes reportFailure within same transaction; preserve existing rentals/reservations/assignments but flag affected orders. Healthy telemetry never clears an unresolved restriction. Commit event result once.

Errors / exception result: Duplicate: original result with DUPLICATE disposition. Invalid/stale: no state change. Unverifiable conflict: retryable error or reconciliation result; no business commitment overwritten. SaveFailure: no partial telemetry/incident update.

TelemetryAdapter.ingestParking(source: SourceIdentity, event: TelemetryEvent): EventResult

Purpose: UC-T02/FR-NI-08: reconcile occupancy.

Input and output: Aggregate or space-level observation; returns reconciliation result.

Preconditions: Valid source, configured reporting scope; count within physical capacity; latest Parking state available.

Postconditions / state change: Parking owner compares observation to sessions/reservations. Matching count refreshes time without adding occupancy. Disagreement retains observed/expected values and discrepancy, with disputed capacity not offered. Preserve booking/session states; commit ledger and projections together.

Errors / exception result: Duplicate: no repeated count/alert. Invalid/stale: reject unchanged. Concurrent check-in forces recheck. SaveFailure: no partial release.

TelemetryAdapter.ingestChargingPoint(source: SourceIdentity, event: TelemetryEvent): EventResult

Purpose: UC-T03: reconcile point/session observations.

Input and output: Point condition and optional session payload; returns result.

Preconditions: Valid source and identity; Charging owner verifies point/session/vehicle mapping, schedule and start/stop transition eligibility, including private-EV parking eligibility.

Postconditions / state change: Charging owner records a verified transition and actual time once. Fault invokes reportFailure and preserves unconfirmed session state. Idle does not cancel future holds. Confirmed capacity release invokes Charging's queue policy once within the same commit. Parking remains unchanged.

Errors / exception result: Invalid/stale/duplicate handled consistently. Unmatched or unexpected start produces RECONCILIATION_REQUIRED and a protective owner restriction without inventing a session. Unverified stop cannot free capacity. SaveFailure rolls back.

6. Entity methods

RedistributionOrder.recordArrivals(vehicleIds: Set<String>): void

Purpose: Update local confirmed progress.

Input and output: Nonempty set of newly verified assigned IDs; returns void.

Preconditions: Called by FleetService inside its unit of work; Assigned/InTransit/PartiallyCompleted; every ID belongs and is not already arrived.

Postconditions / state change: Add whole set; calculate PartiallyCompleted or Completed. Local entity method does not write Hub/vehicle state.

Errors / exception result: Invalid subset or transition: no partial local update. Service handles idempotent command retry before this method.

RedistributionOrder.markAtRisk(reason: String): void

Purpose: Retain progress when an outstanding transfer becomes unsafe.

Input and output: Nonempty reason; returns void.

Preconditions: Incomplete order; Assigned/InTransit/PartiallyCompleted/AtRisk.

Postconditions / state change: First call stores resumeStatus and sets AtRisk; subsequent calls update reason without losing resumeStatus or arrived IDs.

Errors / exception result: Completed/Cancelled or missing reason: reject; no physical movement.

RedistributionOrder.transition(target: OrderStatus): void

Purpose: Guard dispatch, risk recovery and cancellation.

Input and output: Target status; returns void.

Preconditions: FleetService verified cross-resource conditions; edge exists in the state rules above.

Postconditions / state change: Change local state only; risk recovery uses resumeStatus and confirmed progress. Completion cannot be forced through this method.

Errors / exception result: InvalidTransition: retain state; no direct transition to Completed or invented reversal of arrival.

IncidentTicket.appendEvidence(evidence: Evidence): Boolean

Purpose: Retain evidence without duplication.

Input and output: Valid identified evidence; returns true when newly appended, false for identical existing evidence.

Preconditions: Ticket not Closed; evidence source verified by calling service.

Postconditions / state change: Append unique evidence and retain its source/time; service separately handles reopening/invalidation if it is new fault evidence.

Errors / exception result: Same ID/different payload: EvidenceConflict; Closed: InvalidTransition.

IncidentTicket.transition(target: IncidentStatus): void

Purpose: Enforce the local incident lifecycle.

Input and output: Target lifecycle state; returns void.

Preconditions: Valid edge in state rules; service has performed repair/owner safety checks before Resolved.

Postconditions / state change: Change status and retain history; no direct resource writes.

Errors / exception result: InvalidTransition: unchanged. Calling this local method alone is never authority to restore a resource.

7. Cross-module agreements and traceability

Fleet queries Hub status, Rental vehicles/commitments, Parking capacity/reconciliation and Charging point/session/schedule status. The same owners apply Fleet-authorized restrictions, movement updates and restoration through explicit in-process contracts. These contracts are proposed integration agreements for owner review, not redefinitions of their classes.

Requirement	Design entry points	Bonus test IDs
UC-F01/F02/F03	getNetworkOverview, inspectVehicle, inspectChargingPoint	TC-FI-01, TC-FI-14
UC-F04 and UC-S03	validateRedistribution, createRedistribution	TC-FI-02, TC-FI-03, TC-FI-04
UC-F05	recordArrival, recordArrivals	TC-FI-02, TC-FI-05, TC-FI-06
UC-I02/I03; abstract UC-I01	reportFailure, updateIncident, resolveIncident, closeIncident	TC-FI-07, TC-FI-08, TC-FI-11
UC-T01 / FR-NI-01/02/07	ingestVehicle, validateEvent, markAtRisk	TC-FI-07, TC-FI-09, TC-FI-12
UC-T02 / FR-NI-08	ingestParking	TC-FI-10
UC-T03 / FR-NI-05	ingestChargingPoint	TC-FI-11
FR-NI-03/06/07	evaluateNetworkHealth, updateOrder	TC-FI-01, TC-FI-12, TC-FI-13
FR-NI-05 atomic resource consistency	All mutation units of work	TC-FI-04, TC-FI-05, TC-FI-06

FR-NI-04/13 expiry stays with Rental/Parking; FR-NI-10/11 scheduling stays with Charging. Fleet communicates verified restrictions/releases to those owners. FR-NI-09/12 belongs to Simulation; the Fleet snapshot contribution is copied and read-only to that consumer.

Fleet & Incident - Test Cases (Bonus)

Owner: Mai Hoang Phuong Dong - 2452265.

These are executable-by-review acceptance test specifications for the proposed design, not claims of passing backend tests. No backend implementation is included in this design submission. Each case starts from isolated fixtures unless a sequence explicitly shares state. The method catalogue defines error and transaction semantics.

Shared fixtures

At 2026-11-08T09:00:00+07:00, OP1 is an authorized Operator, ST1 a Student, and TP1 a telemetry source permitted for the listed resources. Hub A has capacity 20 and physical occupancy 10. Hub B has capacity 10, physical occupancy 6, and one unrelated incoming hold, leaving 3 transfer spaces. Vehicles V1/V2/V3 are healthy at A, with no rental, reservation or transfer commitment. All observations are current. Unless overridden, staleInterval is 15 minutes, lowVehicleThreshold is 2 and capacityWarningPercent is 85. These values are test configuration, not mandated production thresholds. Each write has a unique requestId and the current entity version; explicit retries reuse the original key and payload.

TC-FI-01 - Dashboard freshness and read-only inspection

Field	Value
Related use case	UC-F01/F02/F03, FR-NI-03
Initial state / preconditions	Authorized OP1. Hub A's accepted observation is 08:45; now is 09:00. Charging point CP1 status source is unavailable. Snapshot all business entities before querying.
Input and steps	Evaluate health at 08:59:59 then 09:00; retrieve overview, V1 details and CP1 details.
Expected response	A is current before the threshold and stale at exactly 15 minutes; accepted timestamp remains visible. CP1 is unavailable, not represented as zero use/free. V1 shows condition, commitments and location.
Expected persisted state	Health projections/alerts may change; dashboard/detail queries change no order, reservation, session, ticket, location or physical occupancy.

TC-FI-02 - Feasible assignment and complete arrival

Field	Value
Related use case	UC-F04, UC-S03, UC-F05, FR-NI-05
Initial state / preconditions	Shared fixtures; two selected vehicles V1/V2.
Input and steps	Validate A to B; confirm proposal; verify state; then physically verify V1/V2 at B and submit arrival confirmation.
Expected response	Feasible proposal; Assigned order; then Completed order with two confirmed arrivals.
Expected persisted state	After creation: both confirmed locations remain A, occupancy remains A=10/B=6, two transfer commitments and two incoming holds exist. After arrival: locations B, occupancy A=8/B=8, transfer commitments/holds consumed; unrelated hold remains. Resource availability is recalculated once.

TC-FI-03 - Insufficient destination capacity or conflicting vehicle

Field	Value
Related use case	UC-F04, UC-S03
Initial state / preconditions	Variant A: B occupancy=9 with one incoming hold, so no free space. Variant B: original fixture but V2 has an active reservation.
Input and steps	Validate V1/V2 transfer for each variant; attempt creation with the rejected proposal. In B, remove V2 and revalidate the V1-only proposal.
Expected response	Capacity reason for A; V2 commitment reason for B. Rejected proposals cannot create an order. Revised V1-only selection can pass a new validation.
Expected persisted state	Rejected attempts create no order/hold and change no physical state. No vehicle is silently dropped from a submitted confirmed request.

TC-FI-04 - Concurrent commitment after validation

Field	Value
Related use case	UC-F04, UC-S03, FR-NI-05
Initial state / preconditions	OP1 has a feasible proposal for V1 at version r1.
Input and steps	Before creation, commit a Rental reservation for V1 (version r2); submit the old proposal. Also run two creation commands for V1 against the same initial state under the shared lock.
Expected response	Old proposal receives Conflict/revalidation required. In the race, at most one competing commitment succeeds.
Expected persisted state	No resource is simultaneously assigned and reserved incompatibly. Losing requests create no order/hold; occupancy is unchanged.

TC-FI-05 - Partial arrival, retry and mismatched confirmation

Field	Value
Related use case	UC-F05, FR-NI-05
Initial state / preconditions	Assigned order O1 contains V1/V2; B holds two transfer spaces.
Input and steps	Confirm only V1 using request K1. Retry K1 with identical payload/old expectedVersion. Submit a new command that incorrectly includes already-arrived V1 or unassigned V3. Finally confirm only V2 with current version.
Expected response	First result PartiallyCompleted; identical retry returns original result. New invalid subset is rejected. Final valid confirmation returns Completed.
Expected persisted state	After K1 and its retry: V1 at B, V2 at A; occupancy A=9/B=7; exactly one remaining transfer hold. Invalid subset has no effects. Final totals A=8/B=8.

TC-FI-06 - Atomic failure and lost acknowledgement

Field	Value
Related use case	UC-F05, UC-I02, FR-NI-05
Initial state / preconditions	O1 assigned V1/V2. Separately, ticket I1 is ready for restoration. Enable controlled failure before transaction publication.
Input and steps	Stage vehicle movement, then fail before Hub/order publication. Repeat with failure between staged ticket resolution and owner restriction release. Retry after removing the failure. In another variant, commit successfully but drop the response; retry the same key.
Expected response	Failed publication returns retryable failure, never success. Successful retry commits once. Lost-response retry returns the original result.
Expected persisted state	Failed attempts preserve complete before-state including holds/restrictions. No partial movement, falsely resolved ticket or duplicate occupancy/eligibility effect.

TC-FI-07 - Repeated fault evidence and transfer risk

Field	Value
Related use case	UC-T01, UC-I02, FR-NI-01/02/07
Initial state / preconditions	V1 is in an Assigned order O1; no open fault ticket.
Input and steps	Send TP1 event E1 at 09:01 reporting motor fault. Retry identical E1. Send newer E2 at 09:02 for the same faultKey. Try to record arrival while O1 is AtRisk.
Expected response	First event applied; retry returns original result without new effects. E2 adds evidence to the same ticket. AtRisk arrival is rejected pending review.
Expected persisted state	One equivalent ticket, two distinct evidence records, vehicle restricted and O1 AtRisk with resumeStatus=Assigned. Original confirmed location and both Hub occupancies remain unchanged.

TC-FI-08 - Incident repair, guarded restoration and recurrence

Field	Value
Related use case	UC-I02; IncidentTicket lifecycle
Initial state / preconditions	Reported I1 on V1 with an affected reservation.
Input and steps	ACCEPT, PLAN and START_REPAIR. Request restoration without completed repair: reject. Record completed inspection but leave affected reservation unresolved: reject. Record verified owner response and make current owner state safe; resolve and close. In a separate branch, send a newer matching fault after Resolved but before Closed.
Expected response	RepairIncomplete then UnresolvedCommitment; later Resolved and Closed. A recurrence before closure reopens Investigating and requires fresh recovery evidence.
Expected persisted state	Vehicle restricted on both failed restorations. Successful resolution removes only I1's restriction, preserving any valid compatible assignment/booking. Recurrence invalidates old repair clearance and restricts the vehicle again. A new fault after Closed starts a new ticket.

TC-FI-09 - Invalid, stale and conflicting telemetry

Field	Value
Related use case	UC-T01, FR-NI-01/03
Initial state / preconditions	V1's last accepted observation is E2 at 09:02; V1 remains restricted by a fault.
Input and steps	Submit older healthy event at 09:01, a different event at equal timestamp 09:02, battery=101, unknown resource, unauthorized provider and reused E2 ID with changed content. Then submit a fresh healthy event at 09:03.
Expected response	Invalid/stale/unauthorized/conflicting events rejected with reasons. Fresh valid observation applied without incident restoration.
Expected persisted state	Rejections change neither accepted observation time nor condition/commitments. Healthy observation updates reported condition only; unresolved incident restriction remains.

TC-FI-10 - Parking observation does not double-count or check out

Field	Value
Related use case	UC-T02, FR-NI-08
Initial state / preconditions	A has occupancy=10 including an already checked-in vehicle, and two future reservations.
Input and steps	Send aggregate occupiedCount=10; repeat the event. Send a newer count=9 without a matching checkout. Also race a newer observation against a check-in.
Expected response	Matching observation refreshes time; duplicate returns previous result. Disagreement returns RECONCILIATION_REQUIRED with expected/observed counts. Concurrent update is reconciled against current owner state.
Expected persisted state	Count never becomes 20. The count=9 report does not check out a vehicle, release a reservation or expose disputed space. Existing sessions/holds remain until owner reconciliation.

TC-FI-11 - Charging fault, future allocation and session evidence

Field	Value
Related use case	UC-F03, UC-I03, UC-T03, FR-NI-05
Initial state / preconditions	CP1 has active session S1 plus future allocation A2; associated private-EV parking is active.
Input and steps	Send point fault without confirmed stop, then healthy/idle observation. Submit wrong-session stop and unexpected start events. Finally submit verified S1 stop with correct resource/time references; repeat it. Process the fault ticket through repair and restoration using owner checks.
Expected response	Fault incident and point restriction; idle cannot restore it. Mismatched events require reconciliation. Verified stop records its actual outcome once. Restoration requires verified repair and reconciliation, not merely idle telemetry.
Expected persisted state	Fault alone does not fabricate S1 completion or free CP1. Future allocation A2 is preserved/flagged until Charging handles it. Correct stop updates only S1; any queue reevaluation follows Charging policy once, with a still-faulted point excluded. Parking remains active.

TC-FI-12 - Recover or cancel an at-risk partial transfer

Field	Value
Related use case	UC-F04/F05, FR-NI-07; RedistributionOrder lifecycle
Initial state / preconditions	O1 assigned V1/V2; V1 already arrived at B. V2 faults at A; O1 AtRisk, resumeStatus=PartiallyCompleted.
Input and steps	Resolve V2 fault via its incident workflow. REVALIDATE O1 with owner state/capacity checked. Separately, attempt CANCEL while V2 disposition is unknown; then reconcile V2 as remaining at A and cancel.
Expected response	Revalidation returns PartiallyCompleted, never resets to Assigned. Unknown-disposition cancellation rejects. Verified cancellation returns Cancelled.
Expected persisted state	V1 remains at B and its occupancy effect remains. Cancellation releases only V2's outstanding holds, leaves V2's confirmed location at A and derives its availability from remaining owner restrictions.

TC-FI-13 - Imbalance alert does not dispatch vehicles

Field	Value
Related use case	UC-F01, FR-NI-03/06
Initial state / preconditions	B has 1 available vehicle with threshold 2; current reliable data.
Input and steps	Run evaluateNetworkHealth twice at the same state; inspect alerts/orders. Restore availability to 3 with a confirmed owner event and run again. Repeat with stale Hub input.
Expected response	One shortage alert updated, then resolved when current data clears the shortage. Stale data receives a data-quality warning.
Expected persisted state	No duplicate alert, new order, physical transfer or reservation cancellation is produced by the background check.

TC-FI-14 - Access control, closed-ticket guards and snapshot isolation

Field	Value
Related use case	UC-F01/F04/I02, integration with UC-W02 and FR-NI-12
Initial state / preconditions	ST1 is a Student; I1 is Closed. Capture a versioned Fleet snapshot through the shared snapshot contract.
Input and steps	ST1 requests dashboard/redistribution/incident restoration. Submit a Student report for another user's rental. OP1 tries a repair update on I1. Change order/ticket data inside a simulated copy.
Expected response	Protected operations/report are rejected; Closed update rejected. Simulation may change only its own copied state.
Expected persisted state	No unauthorized changes or protected data disclosure. Live orders, tickets, occupancy and commitments equal their pre-simulation values.